Rapport technique

Mon Miam Miam / ZeDuc-Sp@ce

Applications web et mobile • API Laravel • MySQL

6 novembre 2026

État du code, circulation des données, droits, vérification et travaux restants

Date : 6 novembre 2026 **Périmètre :** web public, étudiant web et Expo, employé web et Expo, gérant web, admin restaurant, admin plateforme, Tauri, API Laravel et MySQL. **Branche :** feature/sprint-3-backend-restaurant-operations **Statut :** développement et recette locale ; aucun déploiement de cette livraison.

1. Objet et limites du rapport

Ce rapport décrit le code présent dans le dépôt après l'ajout de l'espace étudiant web et des mises à jour rapprochées entre applications. Il s'appuie sur le cahier des charges structuré, la matrice des responsabilités, les routes API et les tests du backend. Une analyse statique et des tests Feature Laravel ne prouvent pas à eux seuls l'affichage sur navigateur ou téléphone. L'utilisateur conserve le lancement et la compilation des applications.

Le cahier V1.0.2 définissait surtout l'application web et les rôles étudiant, employé, gérant et direction. La séparation admin restaurant / admin plateforme, le multi-restaurant, Expo, Tauri, Kinshasa et le CDF sont des évolutions demandées ensuite. Les règles métier non validées ne doivent pas être présentées comme effectives.

2. Architecture et circulation des données

Le navigateur React/Vite, les clients Expo et la coquille Tauri parlent à la même API Laravel. Sanctum délivre un jeton Bearer ; les routes métier appliquent le rôle, l'activité du compte et l'affectation au restaurant. MySQL conserve restaurants, contenus, paniers, commandes, événements de statut, réclamations, paiements, fidélité, parrainages et notifications. Un worker Laravel sur la file notifications,exports persiste les notifications et prépare les exports.

Chemin d'une commande : étudiant web/Expo -> POST /api/restaurants/{restaurant}/orders avec clé d'idempotence -> transaction Laravel, prix et stock recalculés -> commande received et événement -> notification à l'équipe du restaurant -> liste employé du même restaurant -> transitions versionnées -> liste étudiant, vues gérant/admin et agrégats. Les interfaces relisent le serveur ; elles ne fabriquent pas de confirmation locale en mode connecté.

La mise à jour interactive est actuellement du **polling sécurisé**, pas un canal WebSocket. Étudiant web et mobile relisent les commandes visibles environ toutes les 4 s ; employé web 4 s, employé Expo 5 s, gérant 5 s, admin restaurant 8 s, admin plateforme 10 s. Les onglets web cachés ne lancent pas le polling périodique. Les opérations peuvent paraître plus tard si le réseau, l'API ou la file sont lents. Une transmission réellement instantanée nécessite un transport push et une recette de charge distincts.

3. Comptes et frontières de droits

- **Étudiant** — Inscription web /etudiant ou Expo via /api/auth/register; Catalogue public, son panier, ses commandes, adresses, réclamations, points et intentions de paiement.
- **Employé** — Admin restaurant ou gérant autorisé dans son restaurant; Commandes et préparation, disponibilité/plat du jour, proposition sur réclamation ; adresse de livraison seulement pendant une mission affectée.
- **Gérant** — Admin restaurant ou compte affecté selon les routes d'équipe; Supervision, équipe, affectations de livraison, statistiques et validation des réclamations du restaurant.
- **Admin restaurant** — Création/affectation par admin plateforme; Configuration et décisions de son établissement, sans accès aux données internes des autres.
- **Admin plateforme** — Premier compte créé par commande locale puis invitations autorisées; Fiches d'établissements, affectations d'admins, audit, agrégats, identifiants d'escalade. Aucun détail nominatif interne.

Un compte suspendu est refusé. Une affectation multiple donne un sélecteur de restaurant dans les espaces professionnels ; l'API revalide chaque requête. Aucun rôle livreur séparé n'a été créé : un employé peut être affecté à une livraison. Les jetons des clients web restent en mémoire pendant la session active et ne sont pas placés dans le stockage local par le nouvel espace étudiant.

4. Espace étudiant web livré

La route /etudiant est accessible depuis le portail public /acces. Le style reprend le thème sombre, les panneaux translucides, les accents dorés et les cartes visuelles déjà employés dans l'écosystème. L'écran fonctionne sur largeur étroite et bureau. Il refuse de se présenter comme connecté si VITE_API_URL manque.

Fonctions raccordées : inscription, connexion, déconnexion, demande et saisie de jeton local de réinitialisation ; choix d'un restaurant ; menu publié avec visuels et options ; panier unique par restaurant avec confirmation avant remplacement ; adresse à Kinshasa ; devis serveur ; commande idempotente ; historique et statut ; suivi d'une position datée par lien cartographique ; intention de règlement en espèces ; réclamation, message et pièce jointe privée ; promotions/événements informatifs ; participation enregistrée aux jeux ; solde de fidélité, parrainage et classement ; profil et notifications.

Une commande n'est affichée comme créée qu'après la réponse du serveur. cash_due signifie « espèces à régler » et jamais « payé ». Une promotion visible ne réduit pas automatiquement le devis. La participation à un jeu ne prouve aucun gain. Les créneaux et montants sont validés par Laravel. Les URL d'image du serveur ont priorité ; à défaut, les anciens visuels du projet sont explicitement illustratifs.

5. Opérations professionnelles et statistiques

Le backend conserve les transitions received, accepted, preparing, ready, served, delivering, delivered, cancelled, chacune avec auteur, date, version et motif éventuel. Un saut de transition ou une version périmée renvoie 409. Les commandes d'un autre restaurant ne sont pas visibles à l'employé, au gérant ou à l'admin local. La plateforme n'ouvre aucune commande nominative.

L'employé web affiche maintenant un avis lorsqu'une nouvelle commande received apparaît après son premier chargement. Les vues professionnelles relisent l'API plus souvent. Les statistiques séparent les commandes acceptées (confirmed) et les services terminés (completed : served, delivered, ou historique paid). completed_volume_cdf et completed_daily changent après la transition finale. Ils représentent la valeur des commandes servies/livrées, **pas des fonds encaissés**. La plateforme voit les totaux agrégés correspondants sans identité client, adresse, lignes, motif de réclamation ou paiement.

6. Modèle de données et contrats utiles

Entités principales : restaurants, restaurant_user, restaurant_contents, menu_categories, menu_options, service_slots, dining_tables, delivery_zones, delivery_addresses, carts, cart_items, orders, order_lines, order_events, table_reservations, delivery_positions, complaints, complaint_messages, complaint_attachments, loyalty_rules, loyalty_movements, referral_codes, referrals, game_participations, payment_intents, payment_events, app_notifications.

Les prix sont des entiers CDF. expected_version protège les ressources mutables ; Idempotency-Key protège la création de commande et d'intention de paiement. La réponse 409 impose un rechargement avant nouvelle mutation. Le catalogue public livre uniquement les fiches et contenus publiés, actifs et non expirés. Les clés secrètes de paiement ne vont ni dans React ni dans Expo. Le détail des routes est dans REFERENCE_API.md du même dossier.

7. Paiement, fidélité et jeux

Le moyen normalement utilisable est le règlement en espèces avec intention cash_due. Le webhook signé de test est limité à l'environnement local et au drapeau sandbox. Visa, Mastercard, Airtel Money, M-Pesa, BTC et ETH ne sont pas des moyens encaissables à ce stade. Seul un webhook vérifié peut passer une intention à paid dans ce prototype.

Les points sont attribués dans la transaction de service final si une règle du restaurant a été explicitement validée puis activée. Un événement unique évite le double crédit. Le parrainage est propre au restaurant et récompensé après la première commande éligible. Les jeux enregistrent une participation, sans résultat ni récompense serveur. Le classement expose au plus dix alias calculés à partir des commandes servies/livrées.

8. Sécurité, confidentialité et exploitation

Les routes privées contrôlent le jeton Sanctum, le compte actif, le rôle et le restaurant. Les adresses et pièces jointes restent privées ; l'employé affecté perd l'accès à l'adresse après livraison ou annulation. Les pièces jointes sont sur le disque Laravel local, accessibles par URL signée courte avec jeton et vérification du compte. Les journaux portent X-Request-ID et ne journalisent pas les corps ou paramètres d'URL dans la télémétrie HTTP.

La composition Docker locale comprend MySQL, Laravel, frontend et worker ; la composition de production préparée sépare MySQL privé, PHP-FPM, queue et passerelle HTTPS. Le guide RECETTE_PRODUCTION.md décrit migrations, sauvegarde et restauration. Les certificats, secrets, DNS, restauration réelle et essais MySQL restent à vérifier manuellement avant toute production. Aucun déploiement n'a été effectué dans cette livraison.

9. Vérification réalisée et recette manuelle

Un test Feature CrossAppOrderFlowTest traverse une commande étudiante, la lecture par l'employé du bon restaurant, quatre transitions, l'indicateur gérant et l'agrégat plateforme. Il vérifie aussi le refus du restaurant B et l'absence de nom client dans la projection plateforme. Les tests de statistiques et de livraison contrôlent les nouveaux compteurs avant/après statut final et la confidentialité des adresses. La suite Feature Laravel a passé **51 tests et 627 assertions** le 6 novembre 2026. L'analyse ESLint des nouveaux composants a également passé. Aucun lancement ni compilation d'application n'a été effectué.

Pour une recette manuelle, créer d'abord un compte plateforme local via php artisan mmm:create-platform-admin, puis deux restaurants et leurs créneaux ; affecter un admin local, un gérant et un employé au restaurant A, un employé séparé au B ; inscrire un étudiant depuis /etudiant. Démarrer soi-même Docker, MySQL, API, queue, Vite et Expo selon LANCEMENT.md et RECETTE_PRODUCTION.md. Vérifier les badges « Connecté/API », puis :

- Admin A publie un plat. Le site public et l'étudiant web/Expo le voient dans A, jamais dans B.
- Étudiant commande avec devis et clé d'idempotence. La même clé ne crée pas une deuxième commande.
- Employé A voit l'avis de nouvelle commande et la liste ; employé B ne voit aucun détail de A.
- Employé A avance la commande. Étudiant web et Expo voient le statut après synchronisation ; le gérant voit les opérations locales.
- Au passage served/delivered, gérant et admin A voient completed et completed_volume_cdf augmenter ; la plateforme voit seulement leur agrégat.
- Créer une réclamation, traiter employé -> gérant -> admin, puis vérifier la projection étudiante et l'escalade minimale plateforme.
- Créer une intention espèces ; vérifier cash_due, sans « payé ». Tester le webhook signé uniquement dans la sandbox locale.
- Suspendre un compte et retirer une affectation ; vérifier refus immédiat à la prochaine requête et disparition des données d'un autre restaurant.

Le plan de recette détaillé à quinze scénarios est dans ETAT_FRONT_BACK_ET_TESTS_INTERAPPS.md.

10. Ce qui manque encore

Priorité haute : un transport push (Laravel Reverb/WebSockets ou équivalent) pour un vrai instantané, plus une mesure de latence et des reconnexions ; une recette MySQL et une vérification visuelle sur web, Expo et Tauri ; pagination complète au-delà de 100 éléments ; upload et service des médias des plats/événements ; meilleure gestion des rechargements concurrentiels et des erreurs réseau sur chaque écran.

Produit : paiement électronique sous contrat/sandbox validés ; règles de remise des promotions ; validation des gains de jeux ; inscription aux événements ; textes légaux et coordonnées actives ; politique de géolocalisation/conservation des positions ; éventuellement commentaires après commande si confirmés. Le jeu web enregistre une participation, mais ne reproduit pas encore toutes les animations et scores locaux du mobile. Les statistiques employé détaillées restent non définies et donc non exposées en mode réel.

Front : édition complète des catégories/options/allergènes et détails de campagnes dans l'admin connecté ; gestion de la fiche publique et des politiques ; notifications professionnelles visibles dans chaque espace ; suivi cartographique intégré sur le web étudiant (lien cartographique actuellement), pagination et contrôle de rafraîchissement ; davantage de tests automatisés des composants et des parcours UI. La coquille Tauri a encore un écran de connexion de démonstration avant l'entrée dans les espaces ; la vérification de rôle réelle se fait dans les applications embarquées.

Avant publication : terminer le plan de recette, tester les restaurations et sauvegardes sur MySQL, vérifier HTTPS/CORS avec le vrai domaine, charger les vrais contenus et médias des restaurants, puis décider et documenter les règles métier. Les comptes et restaurants fictifs ne doivent pas être utilisés comme données de production.

11. Références internes

- docs/CAHIER_DES_CHARGES.md : synthèse des exigences initiales.
- docs/sprint-3/03-administration/RESPONSABILITES_PAR_ROLE.md : frontière des responsabilités.
- docs/sprint-3/09-integration-recette/REFERENCE_API.md : inventaire des routes.
- docs/sprint-3/09-integration-recette/ETAT_FRONT_BACK_ET_TESTS_INTERAPPS.md : écarts et tests entre applications.
- docs/sprint-3/09-integration-recette/RECETTE_PRODUCTION.md : commandes de recette, migration et sauvegarde.
- docs/sprint-3/09-integration-recette/PAIEMENTS_API.md : états de paiement et webhook local.

12. Lancement simultané et contrôle du Sprint 4

Le lanceur Windows scripts/start-ecosystem.ps1 démarre en une commande Docker Compose pour MySQL, l'API Laravel, la file de notifications et le web, puis ouvre deux terminaux interactifs pour Expo SDK 58 et Tauri. Le serveur Vite déjà exposé sur le port 5173 est réutilisé par Tauri, ce qui évite le conflit entre le site web et la fenêtre desktop. Si Tauri est lancé seul, son script ensure-vite.mjs démarre Vite à sa place. Le mobile utilise l'adresse IPv4 locale du PC fournie au lanceur ; le téléphone doit partager le réseau local.

Commande de précontrôle : ./scripts/start-ecosystem.ps1 -CheckOnly -LanIp <IP_DU_PC>. Commande de lancement : ./scripts/start-ecosystem.ps1 -LanIp <IP_DU_PC>. Le lanceur attend une réponse positive de /api/ready et de desktop.html, puis exécute un contrôle réseau avant d'ouvrir les deux autres applications. Le même contrôle, Expo compris, peut être rejoué avec ./scripts/test-ecosystem.ps1. APP_KEY est générée dans le fichier .env local si elle est vide. Arrêt : Ctrl+C dans les fenêtres Expo et Tauri, puis ./scripts/stop-ecosystem.ps1 pour Docker. Aucun volume MySQL n'est supprimé.

Contrôles effectués dans cette livraison : analyse syntaxique des scripts PowerShell, analyse syntaxique JavaScript et JSON Tauri, docker compose config --quiet, et exécution du précontrôle sans serveur lancé. Ce précontrôle signale que APP_KEY est vide et que Node 23 est installé sur ce poste ; le lanceur génère la clé au démarrage réel, tandis que Node 22.13+ ou 24.3+ est préférable pour Expo SDK 58. La suite Feature Laravel repasse avec **51 tests et 627 assertions**. npm run lint passe avec **zéro erreur et trois avertissements** relatifs aux dépendances de hooks préexistants. La recette visuelle sur les trois surfaces et le test réel sur MySQL restent à faire après lancement par le propriétaire.